



**St. Xavier's College
(Autonomous), Kolkata**

Automated Resume Screening

Arya Barman (506)

Srijani Dutta (516)

Pameli Sensharma (536)

Under supervision of Dr. Debabrata Datta

**Submitted to the Department of Computer Science in partial
fulfilment of the requirements for the degree of M.Sc.**

CERTIFICATE OF AUTHENTICATED WORK

This is to certify that the project report entitled “**Automated Resume Screening**”, submitted to the Department of Computer Science St. Xavier’s College, Kolkata, in partial fulfilment of the requirement for the award of the degree of Masters of Science (M.Sc.) is an original work carried out by:

Name	Roll No	Registration No
Arya Barman	506	A01-2122-0051-24
Srijani Dutta	516	A01-2142-0933-21
Pameli Sensharma	536	

under my guidance. The matter embodied in this project is authentic and is genuine work done by the student and has not been submitted whether to this College or to any other Institute for the fulfilment of the requirement of any course of study.

.....
Signature of the Supervisor

.....
Signature of the Head of the Department.

Dr. Debabrata Datta
Name of the Supervisor

Date:

Date:

Name and Signature of the Project Team Members:

NAME	SIGNATURE
1.	
2.	
3.	

ACKNOWLEDGEMENT

We humbly express our gratitude towards our,

Principal: Rev. Dr. Dominic Savio, S.J.
Head of the Department: Dr. Romit Beed
Mentor: Dr. Debabrata Datta

And also, we all team members are humbly grateful for each other's contribution as without our united work, this would not have been possible.

We acknowledge the contribution of everyone for their continuous support and encouragement throughout our undergraduate studies. This would not have been possible without all of you.

ABSTRACT

In today's competitive job market, organizations receive a large number of applications for a single position, making resume screening a time-consuming and inefficient task. This project presents an Automated Resume Screening System designed to assist recruiters in identifying the most relevant candidates quickly and effectively. The system aims to streamline the initial phase of recruitment by automating the evaluation and shortlisting of resumes based on job requirements.

The proposed system analyses resumes by extracting key information such as skills, qualifications, and experience, and compares them with the given job description to determine candidate suitability. By doing so, it ensures a more consistent, unbiased, and efficient screening process, reducing the dependency on manual efforts and minimizing human error.

The system incorporates natural language processing techniques for preprocessing textual data and uses machine learning algorithms such as Naïve Bayes and Logistic Regression for classification. The models are trained to categorize resumes into relevant job roles and are evaluated using metrics like accuracy and F1-score, along with confusion matrix analysis to assess performance.

Overall, the system provides a scalable and practical solution for modern recruitment challenges, enabling faster hiring decisions, improved candidate matching, and better utilization of organizational resources.

CONTENTS

SL. NO.	TOPICS	PAGE NO.
	Project Certificate	i
	Acknowledgement	ii
	Abstract	iii
1.	Introduction	1-2
1.1.	Background	1
1.2.	Objective	1
1.3.	Purpose	1
1.4.	Scope	2
1.5.	Applicability	2
1.6.	Achievements	2
2.	Survey of Technologies	3-5
2.1.	Machine Learning and NLP technologies	3
2.2.	Web Integration Technologies	4
3.	Requirements and Analysis	6-8
3.1.	Problem Definition	6
3.2.	Requirements Specification	6
3.2.1.	Functional Requirements	6
3.2.2.	Non-Functional Requirements	7
3.3.	Hardware and Software Requirements	8
3.3.1.	Hardware Requirements	8
3.3.2.	Software Requirements	8
4.	System Design	9-18
4.1.	Basic Modules	9
4.2.	Conceptual Models	14
4.2.1.	Flowchart	14

4.2.2.	ER Diagrams	16
4.3	Data Design	18
4.3.1	Schema Design	18
4.3.2	Data Integrity and Constraints	18
5.	Implementation and Testing	19-48
5.1.	Coding details	19
5.1.1.	Environment Setup and Data Loading	19
5.1.2.	Text Preprocessing	21
5.1.3.	Feature Engineering: TF-IDF Vectorisation	22
5.1.4.	Model Training	23
5.1.5.	Model Prediction	31
5.1.6.	Resume Matching and Ranking	33
5.1.7.	Display Results	36
5.1.8.	Web Integration	41
5.2.	Testing	47
5.2.1.	Unit Testing	48
5.2.2.	Integrated Testing	48
6.	Conclusion	49-51
6.1.	Limitations	49
6.2.	Future Scope of the project	50
	References	52-53

1. INTRODUCTION

The **Automated Resume Screening System** is a machine learning-based application developed to simplify the recruitment process. It analyses candidate resumes and compares them with job descriptions to identify relevant skills and qualifications. Using Natural Language Processing techniques, the system evaluates and ranks candidates based on their suitability for a given role. This approach helps reduce manual effort and improves the efficiency and accuracy of candidate selection.

1.1: Background:

In today's world company receives hundreds and sometime thousands of resumes for one single job openings. So, going through each one of the resumes not only time consuming but also inconsistency. That's why technology comes here.

Resume screening project helps to simplify this situation, by speeding up the initial hiring process using Machine Learning and Natural Language Processing technique. Without spending hours on manual screening recruiters can choose perfect candidates for their job roles by matched keywords, relevant skills, experience.

1.2: Objectives:

The main goal of this project is to identify eligible candidate's resume for their matched job roles. The objectives include:

1. Automate the resume screening project.
2. Save time for recruiters by reducing manual efforts.
3. Shortlisting candidates' accuracy improvement.
4. To extract useful information like skills, education, experience to shortlist candidates.
5. To rank candidates' resumes who is most suitable according to the job description.

1.3: Purpose:

The purpose of this project is to develop an Automated Resume Screening System that leverages Machine Learning and Natural Language Processing techniques to simplify the recruitment process. The system is designed to analyse and compare candidate resumes with job descriptions, identify relevant skills and qualifications, and rank candidates based on their suitability. This helps reduce manual effort, minimize bias, and improve the overall efficiency and accuracy of candidate selection.

1.4: Scope:

The scope of this project includes processing and analysing textual data from resumes and job descriptions to extract meaningful information such as skills, experience, and keywords. The system focuses on computing similarity scores and ranking candidates accordingly. It is designed to handle multiple resumes simultaneously and can be extended to different job roles, domains, and larger datasets with further enhancements.

1.5: Applicability:

The proposed Automated Resume Screening System can be applied across a wide range of recruitment and talent acquisition scenarios. It is particularly useful for organizations, HR departments, and recruitment agencies that handle large volumes of job applications, where manual screening becomes time-consuming and inefficient. The system assists recruiters by quickly identifying and shortlisting the most relevant candidates based on job requirements, thereby improving productivity and decision-making.

In addition, the system can be integrated into online job portals and enterprise recruitment platforms to automate the initial stages of hiring. It is also beneficial for campus recruitment drives and consultancy firms that need to evaluate multiple candidates in a limited time frame. With further enhancements, the system can be adapted to different industries and job roles, making it a scalable and flexible solution for modern, data-driven hiring processes.

1.6: Achievements:

Several outcomes have been achieved through this project:

1. A functional automated resume screening system has been successfully developed and implemented.
2. Text preprocessing and feature extraction techniques have been effectively applied to process and analyse resume data.
3. A classification system has been built to categorize resumes based on different job roles.
4. The time required for manual resume filtering has been significantly reduced.
5. Practical knowledge and hands-on experience in Machine Learning and problem-solving have been successfully gained through the development of this project.

2. SURVEY OF TECHNOLOGIES

The development of the Automated Resume Screening System involves the integration of multiple technologies from the domains of Machine Learning, Natural Language Processing, and Web Development. These technologies work together to enable efficient data processing, intelligent decision-making, and user interaction through a web interface.

2.1: Machine Learning and NLP Technologies:

The core functionality of the Automated Resume Screening System relies on Machine Learning (ML) and Natural Language Processing (NLP) techniques for analysing and matching textual data. The following technologies and methods are utilized:

1. **Natural Language Processing:** NLP techniques are used to process and understand textual data from resumes and job descriptions. Tasks such as tokenization, stop-word removal, and lemmatization help in cleaning and structuring the data for further analysis.
2. **Text Preprocessing Techniques:** Preprocessing ensures uniformity in the data by converting text to lowercase, removing punctuation, and eliminating irrelevant words. This step improves the quality and relevance of extracted features.
3. **Feature Extraction (TF-IDF):** Term Frequency–Inverse Document Frequency (TF-IDF) is used to convert textual data into numerical vectors. It helps in identifying important words in a document relative to a collection of documents.
4. **Similarity Measurement:** Cosine similarity is commonly used to measure the similarity between resume vectors and job description vectors. It provides a numerical score indicating how closely a resume matches the job requirements.
5. **Machine Learning Models:** Depending on the implementation, models such as Logistic Regression, Naïve Bayes, or Support Vector Machines (SVM) may be used for classification or ranking tasks.
6. **Libraries and Tools:**
 - Python as the primary programming language.
 - Libraries such as **NumPy**, **Pandas** for data handling.
 - **Scikit-learn** for machine learning models and vectorization.
 - **NLTK** or **spaCy** for NLP tasks.

Advantages:

- Enables automated and intelligent resume evaluation
- Reduces manual effort and human bias in screening
- Handles large volumes of unstructured textual data efficiently
- Provides consistent and accurate matching of resumes with job descriptions
- Improves over time with better data and model tuning
- Supports scalable and data-driven decision-making

2.2: Web Integration Technologies:

To make the system user-friendly and accessible, a web-based interface developed for interaction with the model. The following technologies are used:

1. **Frontend Interface:** The frontend provides a simple interface for users to upload resumes and input job descriptions. Tools such as HTML, CSS, and basic UI frameworks are used to design an interactive layout.
2. **Backend Framework:** Python-based frameworks such as **Flask** or **Streamlit** are used to connect the frontend with the Machine Learning model. These frameworks handle user requests, process inputs, and return results.
3. **File Handling and Parsing:** Libraries such as **PyPDF2** or **docx** are used to extract text from uploaded resume files (PDF/DOCX formats), enabling further processing.
4. **Integration with ML Model:** The trained model is integrated into the backend, allowing real-time prediction and scoring when the user uploads resume and enters job descriptions.
5. **Data Flow Management:** The system ensures smooth data flow from user input to processing and output display, maintaining efficiency and responsiveness.
6. **Deployment :** The application can be deployed on platforms such as local servers or cloud services, making it accessible to users through a web browser.

Advantages:

- Provides a user-friendly and interactive interface.
- Enables real-time processing and instant results.
- Simplifies interaction between users and the ML system.
- Supports easy deployment and accessibility through web browsers.
- Allows integration with existing recruitment platforms.
- Enhances overall usability and system efficiency.

The integration of Machine Learning, NLP, and web technologies provides a robust framework for automated resume screening. Each technology contributes specific advantages, resulting in a system that is efficient, scalable, and capable of delivering accurate and reliable results.

3. REQUIREMENTS AND ANALYSIS

3.1: Problem Definition:

In the modern recruitment process, organizations frequently face the challenge of managing a large number of job applications for a limited number of positions. The manual screening of resumes is not only time-consuming but also prone to human bias and inconsistencies, which may lead to the overlooking of suitable candidates. As the volume of applications continues to increase, traditional methods become inefficient and difficult to scale.

The core problem addressed in this project is the need for an automated system that can efficiently analyse and match candidate resumes with specific job descriptions. The system should be capable of extracting relevant information such as skills, qualifications, and experience, and evaluating their alignment with job requirements. Furthermore, it should rank and categorize candidates based on their suitability without requiring significant human intervention.

To achieve the main objective, the following sub-problems are addressed:

- **Resume Data Extraction:** Extracting meaningful textual information from resumes in different formats such as PDF and DOCX.
- **Text Preprocessing:** Cleaning and standardizing raw text data to remove noise and improve data quality.
- **Feature Representation:** Converting textual data into numerical form using techniques such as TF-IDF for further analysis.
- **Similarity Analysis:** Measuring the similarity between resumes and job descriptions to determine relevance.
- **Candidate Ranking:** Assigning scores and ranking candidates based on their suitability for a given job role.
- **Handling Large Data:** Efficiently processing multiple resumes and ensuring scalability of the system.

3.2: Requirements Specification

3.2.1. Functional Requirements:

1. Dataset Requirement

The system utilizes a pre-existing dataset consisting of resumes and job-related information for training the Machine Learning model. The dataset contains structured

and unstructured textual data, including candidate skills, experience, and job categories. This data is used to train and validate the model so that it can accurately analyse and match resumes with job descriptions. The quality and relevance of the dataset play a crucial role in improving the performance and accuracy of the system.

2. Input Handling:

- The system accepts multiple resumes in PDF or DOCX format.
- The system allows the user to input a job description.

3. Data Preprocessing:

- The system extracts text from uploaded resumes.
- The system preprocesses the text by removing stop-words, punctuation, and irrelevant data.

4. Feature Extraction and Analysis:

- The system converts textual data into numerical form using techniques such as TF-IDF.
- The system analyses resumes and job descriptions to identify relevant features.

5. Machine Learning Model:

- The system utilizes a trained Machine Learning model based on the dataset.
- The system computes similarity scores between resumes and job descriptions.

6. Output Generation:

- The system ranks resumes based on their relevance scores.
- The system displays the ranked list of candidates.
- The system highlights the best-matched candidate

3.2.2. Non-Functional Requirements:

1. **Performance:**

The system should process multiple resumes efficiently and generate results within a reasonable time.

2. **Scalability:**

The system should be able to handle increasing numbers of resumes and larger datasets without performance degradation.

3. **Accuracy:**

The system should provide reliable and consistent results based on the trained dataset and implemented algorithms.

4. **Usability:**

The system should have a simple and user-friendly interface for easy interaction.

5. **Reliability:**

The system should operate consistently without failures during execution.

6. Maintainability:

The system should be easy to update, especially for improving models or incorporating new datasets.

7. Security:

The system should ensure that user data and uploaded resumes are handled securely and confidentially.

3.3: Hardware and Software Requirements

3.3.1. Hardware Requirements:

1. Processor: Intel i5 or equivalent
2. RAM: 8 GB
3. Storage: Minimum 256 GB available space
4. System Type: 64-bit operating system
5. Input Devices: Keyboard and mouse
6. Internet Connectivity: Required for downloading datasets and libraries

3.3.2. Software Requirements

The software requirements are as follows:

1. Operating System: Windows Server
2. Web Server: Google Chrome, Mozilla Firefox, Edge
3. Programming Languages:
 - Python for Machine Learning.
 - Frontend: HTML, CSS, JavaScript
 - Backend: Flask
4. Development Environment:
 - Jupyter Notebook (for model training and testing).
 - Visual Studio Code (for development)
5. Libraries and Tools:
 - NumPy and Pandas for data handling.
 - scikit-learn for TF-IDF vectorization and model implementation
 - NLTK for text preprocessing

4. SYSTEM DESIGN

4.1. Basic Modules

Machine Learning Module:

The Machine Learning module constitutes the core component of the proposed Automated Resume Screening System. It is responsible for processing textual data, extracting meaningful features, training predictive models, and performing both classification and ranking of resumes. The implementation follows a structured pipeline consisting of preprocessing, feature extraction, model training, evaluation, and prediction.

1. Text Preprocessing:

The preprocessing stage is essential for transforming raw textual data into a structured format suitable for analysis. A dedicated preprocessing function is implemented to standardize the input data.

The preprocessing operations include:

- Conversion of all text to lowercase to maintain uniformity
- Preservation and normalization of technical keywords (e.g., programming languages and frameworks)
- Removal of punctuation and special characters using regular expressions
- Elimination of stop-words using Natural Language Processing techniques
- Application of lemmatization to reduce words to their base forms

This stage ensures that the input data is clean, consistent, and optimized for feature extraction, thereby improving the performance of subsequent machine learning models.

2. Feature Extraction using TF-IDF :

The system employs the Term Frequency–Inverse Document Frequency (TF-IDF) technique to convert textual data into numerical feature vectors. This method assigns weights to terms based on their importance within a document relative to the entire dataset.

$$TF-IDF(t, d) = \left(\frac{f(t, d)}{\sum f(t, d)} \right) \cdot \log \left(\frac{N}{df(t)} \right)$$

In the implemented system, the TF-IDF vectorizer is configured with a maximum feature limit and supports both unigrams and bigrams. This allows the model to capture not only individual terms but also meaningful word combinations such as “machine learning” or “data analysis”.

The feature extraction process serves as the foundation for both classification and similarity computation tasks.

3. Dataset Handling and Model Training:

The system utilizes a labelled dataset of resumes for training and evaluation purposes. The dataset is divided into training and testing subsets using an 80:20 split ratio. Additionally, stratified sampling is applied to ensure that the distribution of categories remains consistent across both subsets.

To enhance model reliability and generalization, k-fold cross-validation is employed during training. This approach minimizes overfitting and ensures that the model performs consistently across different subsets of data.

4. Machine Learning Models:

To improve classification accuracy and robustness, multiple supervised learning models are implemented and evaluated.

(a) Naïve Bayes Classifier

The Naïve Bayes classifier is a probabilistic model based on Bayes' Theorem, which assumes independence between features.

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

In this system, the Multinomial Naïve Bayes variant is used, which is particularly suitable for text classification tasks involving word frequency features.

Functionality:

The model calculates the probability of a resume belonging to each category and assigns it to the category with the highest probability. It is computationally efficient and performs well with high-dimensional textual data.

(b) Logistic Regression

Logistic Regression is a supervised learning algorithm used for classification tasks. It estimates the probability of a given input belonging to a particular class using a sigmoid function.

$$P(y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Functionality:

The model predicts class probabilities based on input features and selects the class with the highest probability. It provides stable and interpretable results and is effective for linearly separable data.

(c) Linear Support Vector Machine (SVM)

The Linear Support Vector Machine is a discriminative classifier that identifies the optimal hyperplane separating different classes in a high-dimensional feature space.

$$w \cdot x + b = 0$$

Functionality:

The model maximizes the margin between different classes, ensuring better generalization. It is particularly effective for text classification tasks due to its ability to handle sparse and high-dimensional data efficiently.

5. Model Evaluation using Evaluation Metrics:

The performance of the implemented models is evaluated using standard evaluation metrics such as accuracy, classification report, and confusion matrix. Cross-validation techniques are also applied to ensure that the models generalize well to unseen data.

Multiple evaluation metrics are used here:

Accuracy:

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.
$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

While accuracy provides a quick snapshot, it can be misleading in cases of imbalanced datasets. For example, in a dataset with 90% Resume A and 10% Resume B, a model predicting only Resume A will still achieve 90% accuracy but it will fail to identify any Resume B instances. Accuracy is good but it gives a False Positive sense of achieving high accuracy. The problem arises due to the possibility of misclassification of minor class samples being very high. As the datasets might be imbalanced, other metrics are used as well to achieve more accuracy.

Cross-Validation:

Cross-validation is a technique used to check how well a machine learning model performs on unseen data while preventing over fitting. In real hiring systems, new resumes will always arrive. Cross-validation helps estimate how the model will perform on future unseen resumes. Cross-validation accuracy was used to obtain a more reliable and stable evaluation of the resume screening model. Cross-Validation Accuracy is also used in this project because relying on only one test may give misleading results.

Precision:

It measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high. It measures out of resumes selected for a role, how many were actually correct. It is important when you want to avoid irrelevant candidates. Precision helps reduce unsuitable candidate recommendations.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Where:

TP = True Positives

FP = False Positives

Precision helps ensure that when the model predicts a positive outcome, it's likely to be correct.

Recall:

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where, FN = False Negatives

In scenarios like this where when you don't want to miss strong candidates, recall is a key metric. Out of all suitable resumes, how many the model found. Recall ensures qualified candidates are not overlooked.

F1-Score:

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number. A high F1 score means the model performs well on both metrics. Its range is [0,1].

Lower recall and higher precision give us great accuracy but then it misses a large number of instances. More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \left[\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right]$$

It is useful in this project because it balances avoiding wrong selections and not missing good candidates. F1-score is the most suitable metric because it balances precision and recall, which is important in resume screening where both false positives and false negatives matter.

Support:

Support refers to the number of actual occurrences of the class in the dataset. It is the number of instances in each class. Support is used because it shows the number of actual resumes in each category. This helps interpret Precision, Recall, and F1-score correctly and shows whether the dataset is balanced or imbalanced.

Confusion Matrix:

Confusion matrix creates a N X N matrix, where N is the number of classes or categories that are to be predicted. In this project, confusion matrix helps visualize correct and incorrect classifications for each job category and identifies where the model makes mistakes.

6. Resume Classification:

A dedicated prediction function is implemented to classify new resumes. The function preprocesses the input text, transforms it using the trained TF-IDF vectorizer, and applies the selected machine learning model to predict the corresponding job category. This functionality allows the system to automatically categorize resumes based on their content.

7. Resume Ranking using Cosine Similarity:

In addition to classification, the system performs similarity-based ranking of resumes with respect to a given job description. A separate TF-IDF vectorizer is used for this purpose to ensure better representation of textual similarity. After preprocessing and feature extraction, both the resumes and the job description are represented as numerical vectors using the TF-IDF technique. These vectors capture the importance of terms within each document and enable mathematical comparison.

The similarity between each resume and the job description is computed using cosine similarity:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

where A represents the vector corresponding to a resume and B represents the vector corresponding to the job description.

Cosine similarity measures the cosine of the angle between two vectors in a high-dimensional space. The resulting value ranges from 0 to 1, where values closer to 1 indicate a higher degree of similarity. This method is particularly suitable for text-based applications as it focuses on the orientation of vectors rather than their magnitude, making it robust to variations in document length.

In the implemented system, a dedicated TF-IDF vectorizer is used for ranking purposes to ensure a richer vocabulary representation. The similarity score is computed for each resume, and these scores are then used to determine the relevance of candidates with respect to the job requirements.

8. Output Generation:

Once similarity scores are computed, the system proceeds to rank the resumes in descending order based on their scores. This ranking reflects the degree of alignment between each candidate's profile and the job description.

The final output is presented through the web interface, where users can view the ranked list of candidates along with their corresponding scores. This enables recruiters to quickly identify the most suitable applicants without manually reviewing each resume.

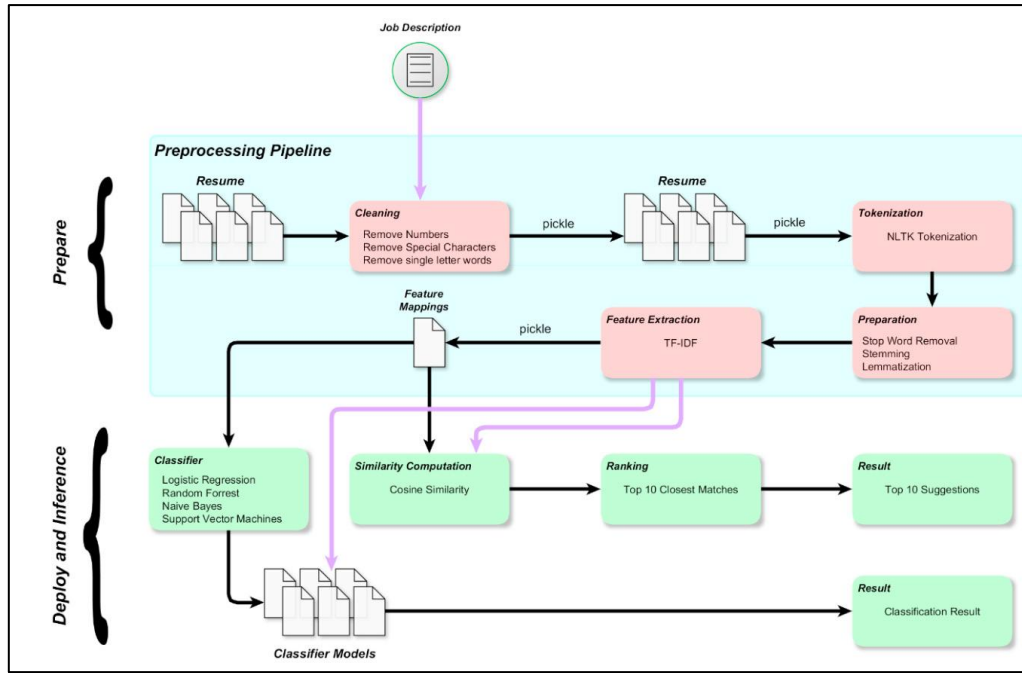


Fig. A complete framework of the proposed model (Reference: A Machine Learning approach for Automated Resume Recommendation System, ICCIDS 2019).

4.2 Conceptual Models

4.2.1. Flowchart

1. The process begins with the Start node.
2. The user uploads resume(s) into the system.
3. The user enters the job description.
4. The system extracts text from the uploaded resume files.
5. The extracted text undergoes preprocessing (lowercasing, stop-word removal, lemmatization).
6. The processed text is converted into numerical form using TF-IDF vectorization.
7. The workflow splits into two parallel processes:
 - Classification: Machine learning models (Naïve Bayes, Logistic Regression, Linear SVM) are applied to predict the resume category.
 - Similarity Ranking: The job description is vectorized and cosine similarity is computed to obtain relevance scores.
8. The outputs from both processes are combined (predicted category and similarity score).
9. The system ranks resumes based on the computed similarity scores.
10. The system displays the results, including ranked candidates and the best match.
11. The process terminates at the End node.

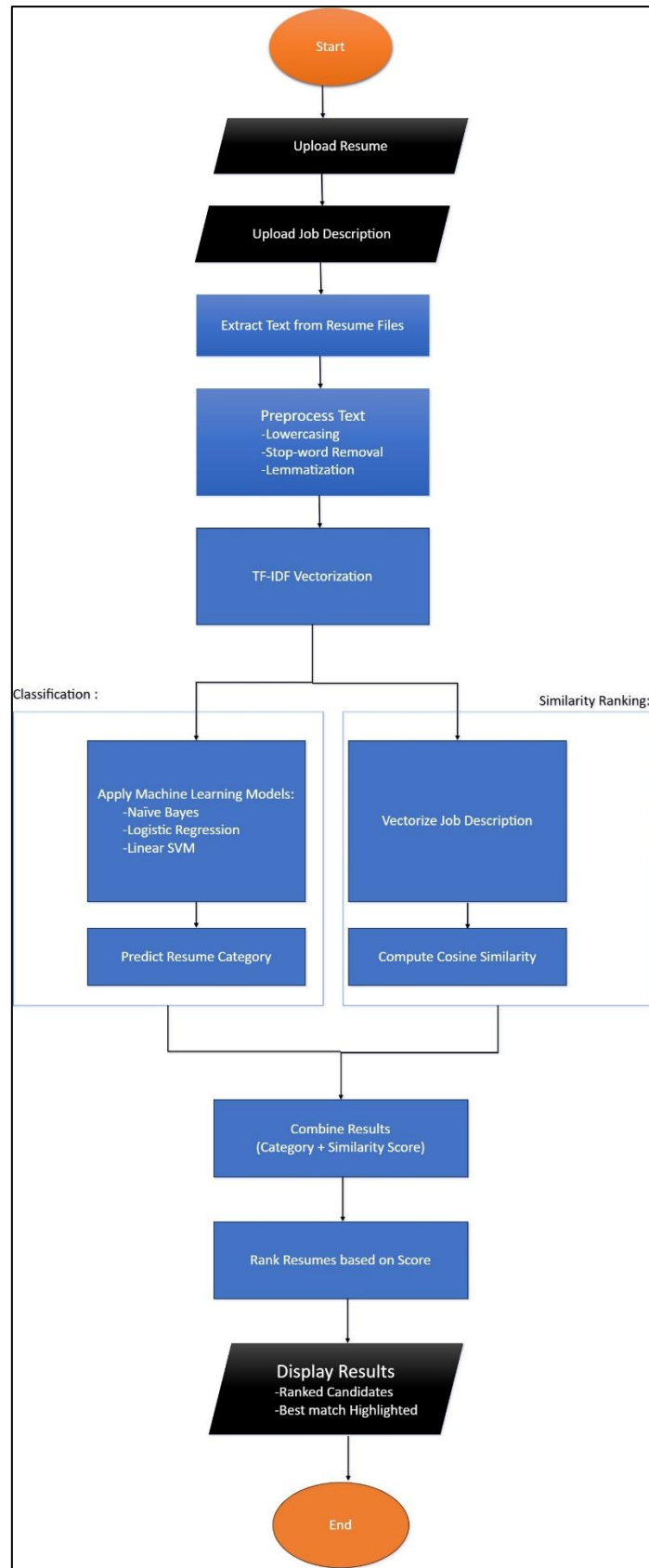


Fig. Flowchart for Automated Resume Screening System.

4.2.2. ER Diagram

The Entity-Relationship (ER) model represents the logical structure of the Automated Resume Screening System by defining the key entities, their attributes, and the relationships between them.

Entities and Attributes:

The system consists of the following primary entities:

1. Resume:

Represents the information extracted from candidate resumes.

Attributes:

- Resume_ID (Primary Key)
- Candidate_Name
- Skills
- Experience
- Education
- Resume_Text

2. Job Description:

Represents the job requirements provided by the user.

Attributes:

- Job_ID (Primary Key)
- Job_Title
- Required_Skills
- Job_Description_Text

3. Classification Result

Represents the predicted category of each resume.

Attributes:

- Classification_ID (Primary Key)
- Resume_ID (Foreign Key)
- Predicted_Category
- Confidence_Score

4. Match Results:

Represents the similarity-based ranking between resumes and job descriptions.

Attributes:

- Match_ID (Primary Key)
- Resume_ID (Foreign Key)
- Job_ID (Foreign Key)
- Similarity_Score
- Rank

Relationships:

8.5.3 Relationships

The relationships between entities are defined as follows:

- A **Resume** can be associated with one **Classification Result** → Relationship: *One-to-One*
- A **Resume** can be matched with a **Job Description** → Relationship: *Many-to-One* (many resumes for one job)
- A **Job Description** can have multiple **Match Results** → Relationship: *One-to-Many*
- Each **Match Result** links a specific resume with a job description → Acts as a bridge entity.

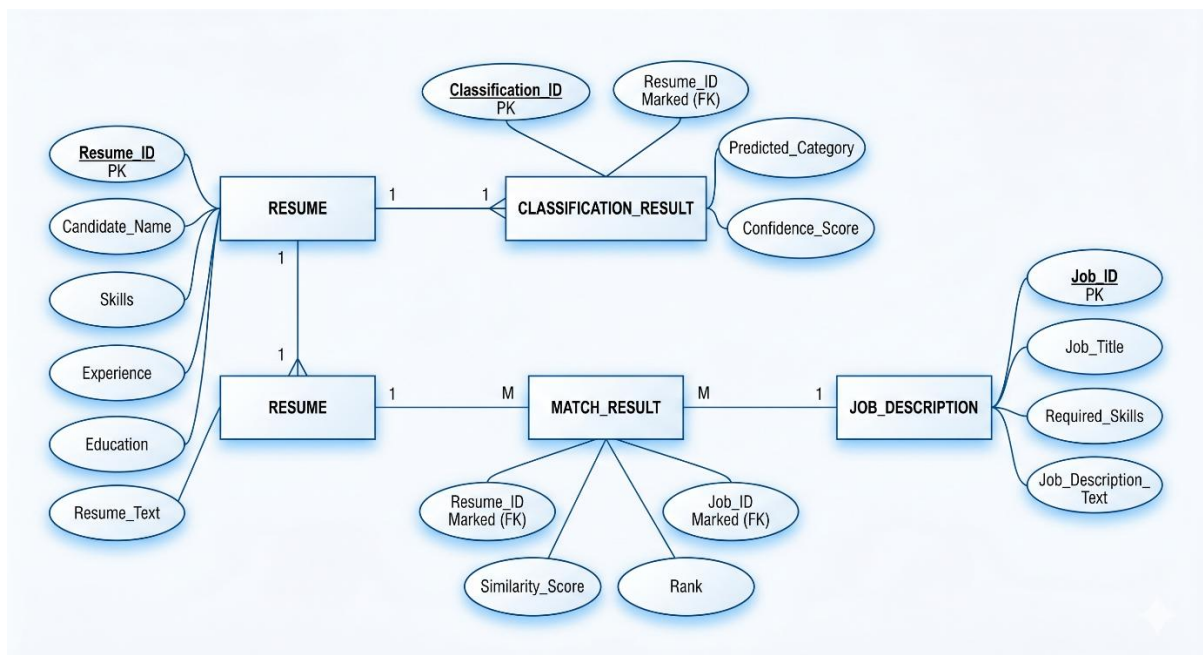


Fig. ER Diagram for the Automated Resume Screening

4.3. Data Design

4.3.1. Schema Design

The schema design defines the structure of data used in the system. The database consists of four primary entities: Resume, Job Description, Classification Result, and Match Result. Each entity is defined with appropriate attributes, along with primary and foreign keys to maintain relationships.

- The Resume entity stores candidate-related information such as skills, experience, and resume text.
- The Job Description entity contains job-related details including required skills and description.
- The Classification Result entity stores the predicted category and confidence score for each resume.
- The Match Result entity acts as a bridge between resumes and job descriptions, storing similarity scores and ranking information.

The schema is designed to support both classification and similarity-based ranking efficiently.

4.3.2. Data Integrity and Constraints

Data integrity ensures the accuracy and consistency of data stored in the system. The following constraints are applied:

- Primary Key Constraints: Each entity has a unique identifier (e.g., Resume_ID, Job_ID) to uniquely identify records.
- Foreign Key Constraints: Relationships between entities are maintained using foreign keys (e.g., Resume_ID in Match Result).
- Not Null Constraints: Important attributes such as Resume_Text and Job_Description_Text cannot be null.
- Data Validation: Input files are validated to ensure they are in supported formats (PDF/DOCX).
- Consistency Constraints: Similarity scores are maintained within a valid range (0 to 1).

These constraints ensure reliable and consistent data processing throughout the system.

5. IMPLEMENTATION AND TESTING

5.1. Coding Details and Efficiency

The Automated Resume Screening System is implemented using Python for data processing and machine learning, along with a web interface for user interaction. The implementation follows a structured pipeline consisting of dataset loading, preprocessing, feature extraction, model training, evaluation, and resume ranking.

The system uses a labeled dataset of resumes (*Resume.csv*) to train classification models and applies cosine similarity for ranking candidates based on job descriptions.

5.1.1. Environment Setup and Data Loading.

All required libraries and downloads the necessary NLTK resources are imported. Standard libraries only are used to ensure reproducibility and portability.

The dataset is loaded from a CSV file. It contains the following columns:

Column	Description
ID	Unique numeric identifier for each candidate
Resume_str	Plain-text version of the resume (used for all ML operations)
Resume_html	HTML-formatted version of the resume (reserved for future UI use)
Category	Target label representing the professional domain

Table: The Resume.csv dataset.

The column Resume_str is used directly as the primary text input for all machine learning operations. The Resume_html column is retained but not used in any ML pipeline steps.

```
# Load the dataset
# Update the file path below if the dataset is located in a different
directory
df = pd.read_csv('Resume.csv')

print(f"Dataset loaded successfully.")
print(f"Total records      : {len(df)}")
print(f"Columns available   : {list(df.columns)}")
print(f"Target categories   : {df['Category'].nunique()} unique categories")
print()
df.head()
```

Output:

```
Dataset loaded successfully.
Total records      : 2484
Columns available  : ['ID', 'Resume_str', 'Resume_html', 'Category']
Target categories  : 24 unique categories
```

	ID	Resume_str	Resume_html	Category
0	16852973	HR ADMINISTRATOR/MARKETING ASSOCIATE\...	<div class="fontsize fontface vmargins hmargin...	HR
1	22323967	HR SPECIALIST, US HR OPERATIONS ...	<div class="fontsize fontface vmargins hmargin...	HR
2	33176873	HR DIRECTOR Summary Over 2...	<div class="fontsize fontface vmargins hmargin...	HR
3	27018550	HR SPECIALIST Summary Dedic...	<div class="fontsize fontface vmargins hmargin...	HR
4	17812897	HR MANAGER Skill Highlights ...	<div class="fontsize fontface vmargins hmargin...	HR

```
# Examine dataset structure and check for missing values
print("Dataset Shape:", df.shape)
print()
print("Missing Values Per Column:")
print(df.isnull().sum())
print()
print("Category Distribution:")
print(df['Category'].value_counts())
```

Output:

```
Dataset Shape: (2484, 4)
```

```
Missing Values Per Column:
```

```
ID          0
Resume_str   0
Resume_html  0
Category     0
dtype: int64
```

```
Category Distribution:
```

```
Category
INFORMATION-TECHNOLOGY    120
BUSINESS-DEVELOPMENT     120
ADVOCATE                  118
CHEF                     118
FINANCE                   118
ENGINEERING               118
ACCOUNTANT                118
FITNESS                   117
AVIATION                  117
SALES                     116
HEALTHCARE                115
CONSULTANT                115
BANKING                   115
CONSTRUCTION              112
PUBLIC-RELATIONS          111
HR                        110
DESIGNER                  107
ARTS                      103
TEACHER                   102
APPAREL                   97
DIGITAL-MEDIA             96
AGRICULTURE               63
AUTOMOBILE                36
BPO                       22
```

```
Name: count, dtype: int64
```

5.1.2. Text Preprocessing.

Raw resume text contains significant noise — including email addresses, URLs, special characters, and numerals — that does not contribute to the semantic meaning relevant for classification. A systematic preprocessing pipeline is defined to normalise all text before feature extraction.

Preprocessing steps applied:

1. Convert text to lowercase to ensure case-insensitive matching.
2. Remove email addresses using a regular expression pattern.
3. Remove hyperlinks and URLs.
4. Remove digits and numeric tokens.
5. Remove special characters and punctuation, retaining only alphabetic tokens.
6. Remove English stop-words using NLTK's standard stop-word list.
7. Apply lemmatisation to reduce inflected words to their base form.

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    # 1. Lowercase
    text = text.lower()

    # 2. Keep technical symbols (C++, C#, .NET) before removing punctuation
    text = re.sub(r'\bc\+\+\b', 'cpp', text)
    text = re.sub(r'\bc#\b', 'csharp', text)
    text = re.sub(r'\.net\b', 'dotnet', text)

    # 3. Remove punctuation but keep alphanumeric characters
    text = re.sub(r'^a-zA-Z0-9\s', ' ', text)

    # 4. Tokenize and Remove Stopwords
    stop_words = set(stopwords.words('english'))
    words = text.split()

    # Filter words: Keep technical terms even if they are short
    # Usually, NLTK removes anything < 3 chars, but we want 'R', 'AI', 'ML'
    cleaned_words = [w for w in words if w not in stop_words or len(w) <= 2]

    # 5. Lemmatization
    lemmatizer = WordNetLemmatizer()
    final_words = [lemmatizer.lemmatize(w) for w in cleaned_words]

    return " ".join(final_words)
```

```
# Apply preprocessing pipeline to the entire dataset
print("Applying text preprocessing to all resumes.")
df['cleaned'] = df['Resume_str'].apply(preprocess_text)

print(f"Preprocessing complete. {len(df)} resumes processed.")
print()
print("Sample – original text (first 200 characters):")
print(df['Resume_str'].iloc[0][:200])
print()
print("Sample – cleaned text (first 200 characters):")
print(df['cleaned'].iloc[0][:200])
```

Output:

```
Applying text preprocessing to all resumes.
Preprocessing complete. 2484 resumes processed.

Sample – original text (first 200 characters):
HR ADMINISTRATOR/MARKETING ASSOCIATE

HR ADMINISTRATOR      Summary      Dedicated Customer Service Manager with 15+ years of experience in Hospitality and Customer Service Management.      Resp

Sample – cleaned text (first 200 characters):
hr administrator marketing associate hr administrator summary dedicated customer service manager 15 year of experience in hospitality customer service management respected builder leader
of customer f
```

5.1.3. Feature Engineering: TF-IDF Vectorisation

Term Frequency-Inverse Document Frequency (TF-IDF) is a well-established numerical representation technique for text data. It assigns higher weights to terms that are frequent within a document but rare across the entire corpus, thereby highlighting discriminative tokens.

Configuration used:

- `ngram_range=(1, 2)` — captures both unigrams and bigrams to retain short multi-word phrases.
- `max_features=5000` — limits the vocabulary to the 5,000 most informative terms to control dimensionality.
- `sublinear_tf=True` — applies logarithmic scaling to term frequencies to reduce the effect of very high-frequency terms.

```
# Encode target labels
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['label'] = label_encoder.fit_transform(df['Category'])

print("Label encoding complete.")
print(f"Number of unique categories: {len(label_encoder.classes_)}")
print(f"Categories: {list(label_encoder.classes_)}")

# — Train / test split —
X = df['cleaned']
y = df['Category']
CLASSES = sorted(y.unique())
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = TEST_SIZE,
    stratify = y,
    random_state = RANDOM_STATE,
)

print(f'Train size : {len(X_train)}')
print(f'Test size : {len(X_test)}')
print(f'Categories : {len(CLASSES)}')

# — Fit TF-IDF —
vectorizer = TfidfVectorizer(
    max_features = TFIDF_FEATURES,
    ngram_range = TFIDF_NGRAM,
    stop_words = 'english',
    sublinear_tf = True,      # log(1+tf) – dampens extreme frequencies
    min_df = 2,              # ignore terms in < 2 documents
    strip_accents = 'unicode',
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print(f'\nTF-IDF matrix – train : {X_train_tfidf.shape}')
print(f'TF-IDF matrix – test : {X_test_tfidf.shape}')

```

Output:

```

Train size : 1987
Test size : 497
Categories : 24

TF-IDF matrix – train : (1987, 5000)
TF-IDF matrix – test : (497, 5000)

```

5.1.4. Model Training

The model training phase involves the application of multiple supervised machine learning algorithms, namely Multinomial Naïve Bayes, Logistic Regression, and Linear Support Vector Machine (SVM), to perform resume classification. The textual data extracted from resumes is first transformed into numerical feature vectors using the TF-IDF (Term Frequency–Inverse Document Frequency) technique. These feature vectors serve as input to the classification models.

To ensure robustness and generalization, the training process incorporates Stratified K-Fold Cross-Validation, wherein the dataset is divided into k subsets while preserving the class distribution across each fold. Each model is trained and validated iteratively across these folds, and the average performance is computed to minimize bias and variance.

For evaluation, Accuracy is used as the primary metric during both cross-validation and final testing. It measures the proportion of correctly classified instances over the total number of predictions. However, recognizing the potential issue of class imbalance in resume datasets, the F1-score is also employed as a complementary metric, providing a balanced measure of precision and recall.

Additionally, a confusion matrix is utilized to perform detailed error analysis by illustrating the distribution of correct and incorrect predictions across different classes. This enables the identification of specific categories where misclassification occurs, thereby offering insights for further model improvement.

The trained model is evaluated on the held-out test set. Accuracy, precision, recall, and F1-score are reported to provide a comprehensive assessment of classification performance across all categories.

```
MODELS = {
    'Naive Bayes'      : MultinomialNB(alpha=0.1),
    'Logistic Regression' : LogisticRegression(
        max_iter=2000, C=1.0,
        solver='lbfgs',
        random_state=RANDOM_STATE),
    'Linear SVM'       : LinearSVC(
        C=1.0, max_iter=3000,
        random_state=RANDOM_STATE),
}

cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True,
random_state=RANDOM_STATE)
results = {}

for name, model in MODELS.items():
    print(f'\n{"-"*55}')
    print(f'  Training: {name}')
    print(f'{"-"*55}')

    # 5-fold CV on training data
    cv_scores = cross_val_score(
        model, X_train_tfidf, y_train,
        cv=cv, scoring='accuracy', n_jobs=-1
    )
    print(f'  CV Accuracy (5-fold) : {cv_scores.mean():.4f} ±
{cv_scores.std():.4f}')

    # Fit on full training set
    model.fit(X_train_tfidf, y_train)
    y_pred = model.predict(X_test_tfidf)
    acc     = accuracy_score(y_test, y_pred)
    print(f'  Test Accuracy      : {acc:.4f}')
```

```

results[name] = {
    'model' : model,
    'y_pred' : y_pred,
    'accuracy': acc,
    'cv_mean' : cv_scores.mean(),
    'cv_std' : cv_scores.std(),
}

print('\nTraining completed.')

```

Output:

Training: Naive Bayes
CV Accuracy (5-fold) : 0.5596 ± 0.0215
Test Accuracy : 0.5775

Training: Logistic Regression
CV Accuracy (5-fold) : 0.6326 ± 0.0113
Test Accuracy : 0.6499

Training: Linear SVM
CV Accuracy (5-fold) : 0.7006 ± 0.0064
Test Accuracy : 0.7384

Training completed.

```

for name, info in results.items():
    print(f'\n{"="*60}')
    print(f' {name} | Test Accuracy: {info["accuracy"]*100:.4f}%')

    print(f'{"="*60}')
    print(classification_report(
        y_test, info['y_pred'],
        target_names=CLASSES,
        zero_division=0
    ))

```

Output:

```

=====
Naive Bayes | Test Accuracy: 57.7465%
=====
precision    recall  f1-score   support

ACCOUNTANT    0.54    0.83    0.66      24
ADVOCATE      0.32    0.38    0.35      24
AGRICULTURE   0.71    0.38    0.50      13
APPAREL       0.71    0.26    0.38      19
ARTS          0.36    0.19    0.25      21
AUTOMOBILE    0.00    0.00    0.00       7
AVIATION      0.83    0.62    0.71      24
BANKING       0.70    0.61    0.65      23
BPO           0.00    0.00    0.00       4
BUSINESS-DEVELOPMENT 0.45    0.79    0.58      24
CHEF          0.86    0.75    0.80      24
CONSTRUCTION  0.65    0.68    0.67      22
CONSULTANT    0.25    0.09    0.13      23
DESIGNER      0.81    0.62    0.70      21
DIGITAL-MEDIA 0.70    0.37    0.48      19
ENGINEERING   0.75    0.62    0.68      24
FINANCE       0.65    0.46    0.54      24
FITNESS       0.61    0.61    0.61      23
HEALTHCARE    0.56    0.61    0.58      23
HR            0.67    0.91    0.77      22
INFORMATION-TECHNOLOGY 0.64    0.96    0.77      24
PUBLIC-RELATIONS 0.42    0.73    0.53      22
SALES         0.44    0.61    0.51      23
TEACHER       0.50    0.70    0.58      20

accuracy              0.58      497
macro avg            0.55    0.53    0.52      497
weighted avg         0.58    0.58    0.56      497

```

```

=====
Logistic Regression | Test Accuracy: 64.9899%
=====
precision    recall  f1-score   support

ACCOUNTANT    0.65    0.83    0.73      24
ADVOCATE      0.37    0.46    0.41      24
AGRICULTURE   1.00    0.31    0.47      13
APPAREL       0.67    0.21    0.32      19
ARTS          0.62    0.24    0.34      21
AUTOMOBILE    0.00    0.00    0.00       7
AVIATION      0.86    0.75    0.80      24
BANKING       0.84    0.70    0.76      23
BPO           0.00    0.00    0.00       4
BUSINESS-DEVELOPMENT 0.55    0.92    0.69      24
CHEF          0.86    0.75    0.80      24
CONSTRUCTION  0.71    0.77    0.74      22
CONSULTANT    0.62    0.22    0.32      23
DESIGNER      0.80    0.76    0.78      21
DIGITAL-MEDIA 0.77    0.53    0.62      19
ENGINEERING   0.74    0.71    0.72      24
FINANCE       0.80    0.67    0.73      24
FITNESS       0.76    0.70    0.73      23
HEALTHCARE    0.53    0.70    0.60      23
HR            0.77    0.91    0.83      22
INFORMATION-TECHNOLOGY 0.71    1.00    0.83      24
PUBLIC-RELATIONS 0.52    0.77    0.62      22
SALES         0.39    0.57    0.46      23
TEACHER       0.56    0.90    0.69      20

accuracy              0.65      497
macro avg            0.63    0.60    0.58      497
weighted avg         0.66    0.65    0.63      497

```

```

=====
Linear SVM | Test Accuracy: 73.8431%
=====

```

	precision	recall	f1-score	support
ACCOUNTANT	0.75	0.88	0.81	24
ADVOCATE	0.76	0.67	0.71	24
AGRICULTURE	0.89	0.62	0.73	13
APPAREL	0.67	0.42	0.52	19
ARTS	0.67	0.38	0.48	21
AUTOMOBILE	1.00	0.29	0.44	7
AVIATION	0.87	0.83	0.85	24
BANKING	0.78	0.78	0.78	23
BPO	0.00	0.00	0.00	4
BUSINESS-DEVELOPMENT	0.76	0.92	0.83	24
CHEF	0.82	0.75	0.78	24
CONSTRUCTION	0.78	0.82	0.80	22
CONSULTANT	0.82	0.39	0.53	23
DESIGNER	0.87	0.95	0.91	21
DIGITAL-MEDIA	0.72	0.68	0.70	19
ENGINEERING	0.73	0.79	0.76	24
FINANCE	0.78	0.75	0.77	24
FITNESS	0.87	0.87	0.87	23
HEALTHCARE	0.59	0.74	0.65	23
HR	0.84	0.95	0.89	22
INFORMATION-TECHNOLOGY	0.77	1.00	0.87	24
PUBLIC-RELATIONS	0.61	0.77	0.68	22
SALES	0.48	0.57	0.52	23
TEACHER	0.61	0.85	0.71	20
accuracy			0.74	497
macro avg	0.73	0.69	0.69	497
weighted avg	0.74	0.74	0.73	497

Confusion matrices are generated for each model to provide a visual summary of classification errors. Each cell in the matrix indicates the number of resumes from a true category (row) predicted as a given category (column). Off-diagonal values represent misclassifications and help identify categories that the model finds most difficult to distinguish.

```

n_cols = len(CLASSES)
fig_h = max(8, n_cols - 2)
fig_w = max(10, n_cols)

for name, info in results.items():
    cm = confusion_matrix(y_test, info['y_pred'], labels=CLASSES)

    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    sns.heatmap(
        cm,
        annot      = True,
        fmt        = 'd',
        cmap        = 'Blues',
        xticklabels= CLASSES,
        yticklabels= CLASSES,
        ax          = ax,
        linewidths  = 0.4,
    )
    ax.set_xlabel('Predicted Label', fontsize=11)

```

```

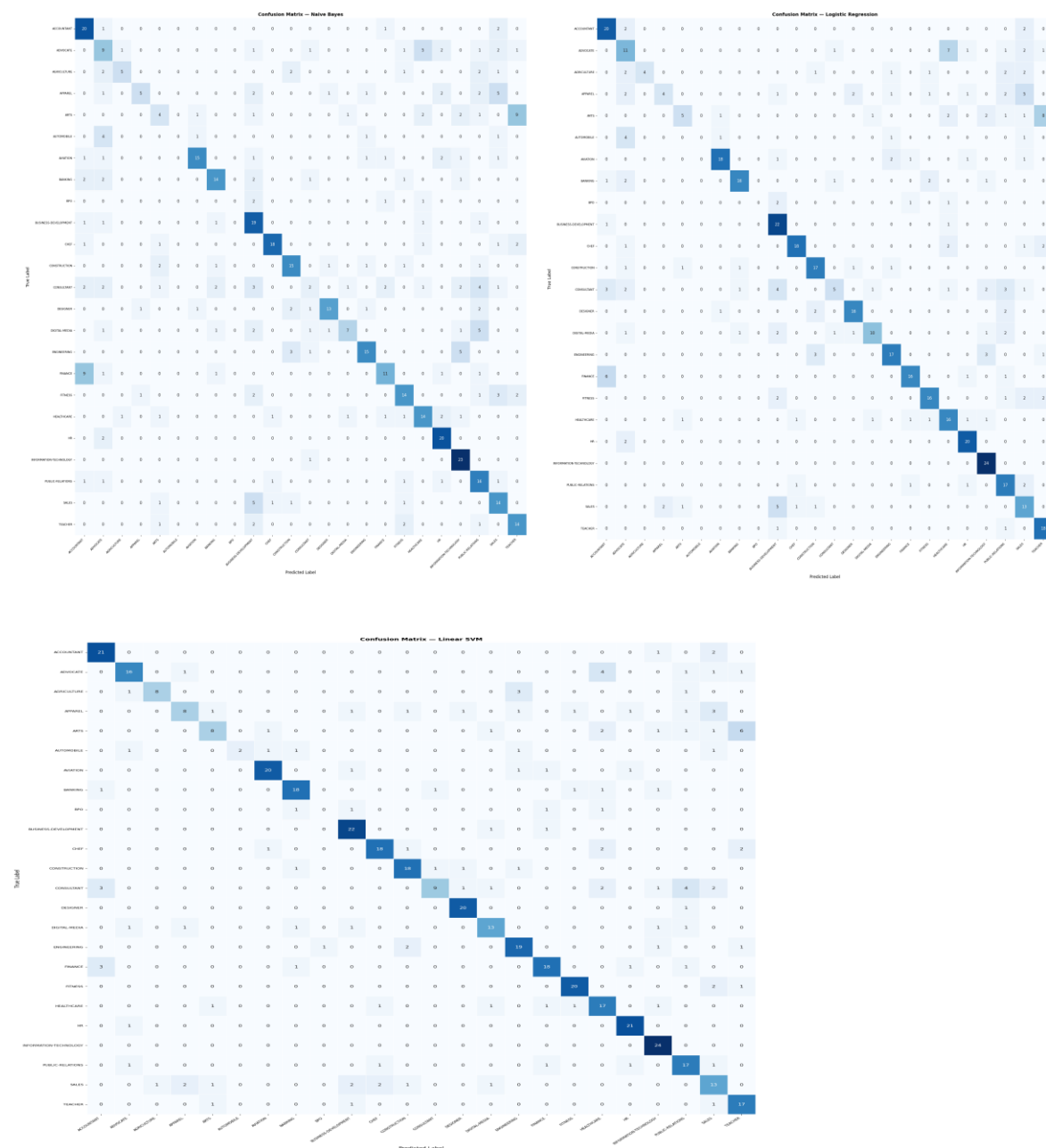
ax.set_ylabel('True Label',      fontsize=11)
ax.set_title(f'Confusion Matrix – {name}', fontsize=13, fontweight='bold')
plt.xticks(rotation=45, ha='right', fontsize=8)
plt.yticks(rotation=0,  fontsize=8)
plt.tight_layout()

safe = name.lower().replace(' ', '_')
plt.savefig(PLOTS_DIR / f'cm_{safe}.png', dpi=130, bbox_inches='tight')
plt.show()

print('Confusion matrices saved to:', PLOTS_DIR)

```

Confusion Matrices for Naïve Bayes, Logistic Regression and Linear SVM:



```

# — Summary table —
summary = pd.DataFrame([
    {
        'Model'      : name,
        'CV Acc Mean': round(info['cv_mean'], 4),
        'CV Acc Std'  : round(info['cv_std'], 4),
        'Test Acc'    : round(info['accuracy'],4),
    }
    for name, info in results.items()
]).sort_values('Test Acc', ascending=False).reset_index(drop=True)

print('='*60)
print('MODEL COMPARISON')
print('='*60)
print(summary.to_string(index=False))
print()

# — Bar chart —
x      = np.arange(len(results))
names  = list(results.keys())
accs   = [results[n]['accuracy'] for n in names]
cvs    = [results[n]['cv_mean']  for n in names]

fig, ax = plt.subplots(figsize=(9, 5))
b1 = ax.bar(x - 0.2, accs, 0.35, label='Test Accuracy', color='#4C72B0')
b2 = ax.bar(x + 0.2, cvs, 0.35, label='CV Accuracy', color='#55A868')
ax.set_xticks(x)
ax.set_xticklabels(names, fontsize=11)
ax.set_ylim(0, 1.05)
ax.set_ylabel('Accuracy')
ax.set_title('Model Comparison – Test vs CV Accuracy', fontsize=13,
fontweight='bold')
ax.legend()
for bar in list(b1) + list(b2):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'model_comparison.png', dpi=130, bbox_inches='tight')
plt.show()

# — Select best model —
BEST_NAME = summary.iloc[0]['Model']
best_model = results[BEST_NAME]['model']

print(f'\nBest model selected : {BEST_NAME} (Test Accuracy =
{results[BEST_NAME]["accuracy"]:.4f})')

```

=====				
MODEL COMPARISON				
=====				
	Model	CV Acc Mean	CV Acc Std	Test Acc
	Linear SVM	0.7006	0.0064	0.7384
	Logistic Regression	0.6326	0.0113	0.6499
	Naïve Bayes	0.5596	0.0215	0.5775

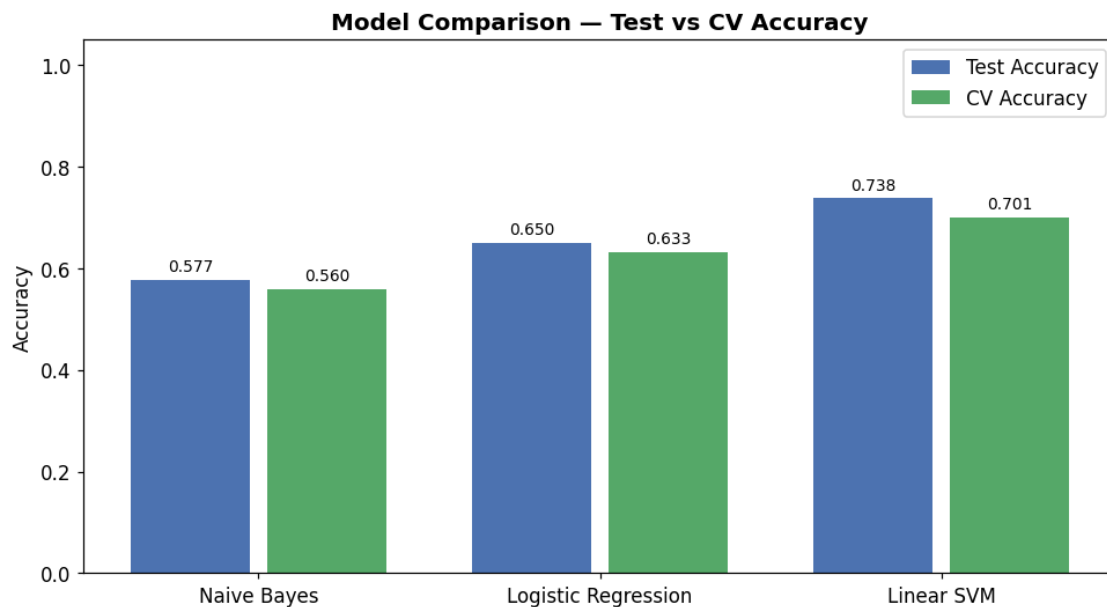


Fig. Model Comparison of Naïve Bayes, Logistic regression and Linear SVM Model.

Best model selected : Linear SVM (Test Accuracy = 0.7384)

```
ranking_vectorizer = TfidfVectorizer(
    max_features = 10000,          # 2× more vocabulary than the classifier
    ngram_range  = (1, 2),        # same unigram + bigram strategy
    stop_words   = 'english',
    sublinear_tf = True,
    min_df       = 1,             # include every term – no cutoff
    strip_accents = 'unicode',
)

ranking_vectorizer.fit(df['cleaned']) # fitted on ALL resumes

print(f"Original classifier vocabulary : {len(vectorizer.vocabulary_):>6,}
features")
print(f"Ranking vectoriser vocabulary :
{len(ranking_vectorizer.vocabulary_):>6,} features")
print("Ranking vectoriser ready.")
```

```
Original classifier vocabulary : 5,000 features
Ranking vectoriser vocabulary : 10,000 features
Ranking vectoriser ready.
```

5.1.5. Prediction

A prediction function is defined to classify a single resume text string into one of the known professional categories. This function applies the same preprocessing pipeline and TF-IDF transformation used during training to ensure consistency.

```
def predict_category(resume_text, model=None, vec=None):
    """
    Predict the job category of a single resume string.

    Parameters
    -----
    resume_text : str    Raw resume text.
    model       : fitted classifier (uses best_model from this session if
None).
    vec         : fitted TF-IDF    (uses vectorizer from this session if
None).

    Returns
    -----
    dict {'category': str, 'confidence': float | None}
    """
    _model = model if model is not None else best_model
    _vec   = vec   if vec   is not None else vectorizer

    # Guard
    if not isinstance(resume_text, str) or not resume_text.strip():
        return {'category': 'INVALID_INPUT', 'confidence': None}

    cleaned = preprocess_text(resume_text)
    if not cleaned.strip():
        return {'category': 'EMPTY_AFTER_PREPROCESSING', 'confidence': None}

    vec_input = _vec.transform([cleaned])
    category   = _model.predict(vec_input)[0]

    confidence = None
    if hasattr(_model, 'predict_proba'):
        confidence = round(float(max(_model.predict_proba(vec_input)[0])), 4)

    return {'category': category, 'confidence': confidence}

# — Test the function —
test_cases = [
    df['Resume_str'].iloc[0],          # real resume from dataset
    'Experienced HR Manager skilled in talent acquisition, HRIS, payroll,
compliance.',
```

```

    'Python developer with expertise in Django, REST APIs, PostgreSQL,
    Docker.',
    '',                                     # edge case: empty
    None,                                   # edge case: None
]

print('PREDICT CATEGORY ')
for i, text in enumerate(test_cases, 1):
    result = predict_category(text)
    preview = (str(text)[:70] + '...') if text and len(str(text)) > 70 else
str(text)
    conf_str = f"{result['confidence']:.2%}" if result['confidence'] else
'N/A'
    print(f'\n[{i}] Input      : {preview}')
    print(f'      Category   : {result["category"]}')
    print(f'      Confidence  : {conf_str}')

# Real Label check for row 0
true_label = df['Category'].iloc[0]
pred_label = predict_category(df['Resume_str'].iloc[0])['category']
match = '' if true_label == pred_label else ''
print(f'\n{match} Row 0  True={true_label} | Predicted={pred_label}')

```

Output:

```

PREDICT CATEGORY

[1] Input      :      HR ADMINISTRATOR/MARKETING ASSOCIATE

HR ADMINISTRATOR      ...
    Category   : HR
    Confidence  : N/A

[2] Input      : Experienced HR Manager skilled in talent acquisition, HRIS, payroll, c...
    Category   : HR
    Confidence  : N/A

[3] Input      : Python developer with expertise in Django, REST APIs, PostgreSQL, Dock...
    Category   : ENGINEERING
    Confidence  : N/A

[4] Input      :
    Category   : INVALID_INPUT
    Confidence  : N/A

[5] Input      : None
    Category   : INVALID_INPUT
    Confidence  : N/A

Row 0  True=HR | Predicted=HR

```

5.1.6. Resume Matching and Ranking

This section implements the core ranking module of the system. Given a user-defined job description, each resume in the dataset is assessed for relevance using cosine similarity computed over the shared TF-IDF vector space.

Methodology

1. Preprocess all resumes and the job description using the same pipeline.
2. Transform all text using the fitted TF-IDF vectoriser.
3. Compute cosine similarity between the job description vector and each resume vector.
4. Predict the professional category for each resume.
5. Rank all resumes in descending order of similarity score.

Cosine similarity measures the angular distance between two vectors in the TF-IDF feature space, providing a normalised relevance score independent of document length.

```
def rank_resumes(resume_list, job_description, candidate_ids=None,
top_n=None):
    # — Validate inputs —
    if not isinstance(resume_list, list) or len(resume_list) == 0:
        raise ValueError('resume_list must be a non-empty list of strings.')
    if not isinstance(job_description, str) or not job_description.strip():
        raise ValueError('job_description must be a non-empty string.')

    resume_list = [r if isinstance(r, str) else '' for r in resume_list]

    if candidate_ids is None:
        candidate_ids = [f'Candidate_{i+1}' for i in range(len(resume_list))]
    elif len(candidate_ids) != len(resume_list):
        raise ValueError('candidate_ids length must match resume_list
length.')

    # — Preprocess —
    cleaned_resumes = [preprocess_text(r) for r in resume_list]
    cleaned_jd      = preprocess_text(job_description)

    if not cleaned_jd.strip():
        raise ValueError('Job description became empty after preprocessing.')

    # — Transform with RANKING vectoriser (cosine similarity) —
    jd_rank      = ranking_vectorizer.transform([cleaned_jd])
    resume_rank  = ranking_vectorizer.transform(cleaned_resumes)

    # — Transform with CLASSIFIER vectoriser (category prediction) —
    resume_cls   = vectorizer.transform(cleaned_resumes)

    # — Cosine similarity — 1-D array —
```

```

scores = cosine_similarity(jd_rank, resume_rank).flatten()

# — Zero-out empty resumes —————
for idx, cleaned in enumerate(cleaned_resumes):
    if not cleaned.strip():
        scores[idx] = 0.0

# — Predict professional category for each resume (SVM) —————
predicted_categories = best_model.predict(resume_cls)

# — Identify relevant categories via category centroids —————
# Build the average ranking-vectoriser TF-IDF vector for each
# category, then compute cosine similarity with the JD.
# This avoids trying to classify the JD with the SVM (unreliable).
centroid_sims = {}
jd_dense = np.asarray(jd_rank.todense())

for cat in df['Category'].unique():
    cat_texts = df[df['Category'] == cat]['cleaned'].tolist()
    cat_matrix = ranking_vectorizer.transform(cat_texts)
    centroid = np.asarray(cat_matrix.mean(axis=0))
    sim = cosine_similarity(jd_dense, centroid)[0][0]
    centroid_sims[cat] = float(sim)

sorted_cats = sorted(centroid_sims.items(), key=lambda x: x[1],
reverse=True)
relevant_cats = set(cat for cat, _ in sorted_cats[:3])

print("Top 3 most relevant categories (by centroid similarity):")
for cat, sim in sorted_cats[:3]:
    print(f" {cat:<30s} {sim:.4f}")

# — Demote resumes outside the top-3 relevant categories —————
final_scores = scores.copy()
for i, cat in enumerate(predicted_categories):
    if cat not in relevant_cats:
        final_scores[i] = 0.0

# — Build result DataFrame —————
df_result = pd.DataFrame({
    'candidate_id' : candidate_ids,
    'similarity_score' : np.round(final_scores, 4),
    'predicted_category' : predicted_categories,
    'resume_preview' : [(r[:150] + '...') if len(r) > 150 else r
                        for r in resume_list],
})

df_result.sort_values('similarity_score', ascending=False, inplace=True)

```

```

df_result.reset_index(drop=True, inplace=True)
df_result.insert(0, 'rank', range(1, len(df_result)+1))

if top_n:
    df_result = df_result.head(top_n)

return df_result

print('rank_resumes() defined.')

```

Define Job Description:

```

job_description = """
We are looking for an experienced Data Scientist with a strong background in
machine learning, statistical modelling, and data analysis. The ideal
candidate should have proficiency in Python and R, experience with frameworks
such as scikit-learn, TensorFlow, or PyTorch, and the ability to communicate
complex findings to non-technical stakeholders. Responsibilities include
developing predictive models, performing exploratory data analysis, building
data pipelines, and collaborating with cross-functional teams. Experience with
SQL databases, cloud platforms (AWS, GCP, or Azure), and big data tools such
as Spark is advantageous. A bachelor's or master's degree in Computer Science,
Statistics, Mathematics, or a related field is required.
"""

print("Job description defined.")
print(f"Character count: {len(job_description)}")

```

Execute Ranking:

```

# Prepare inputs directly from Resume_str and ID columns
# Drop rows with missing resume text and align candidate IDs accordingly
valid_df = df.dropna(subset=['Resume_str']).copy()

resume_list = valid_df['Resume_str'].tolist()
candidate_ids = valid_df['ID'].astype(str).tolist()

print(f"Total resumes available for ranking: {len(resume_list)}")
print(f"Executing ranking pipeline...")

# Execute the ranking function
ranked = rank_resumes(resume_list, job_description, candidate_ids)

print("Ranking complete.")

```

```
Total resumes available for ranking: 2484
Executing ranking pipeline...
Top 3 most relevant categories (by centroid similarity):
ENGINEERING          0.1465
CONSULTANT           0.1447
INFORMATION-TECHNOLOGY 0.1421
Ranking complete.
```

5.1.7. Display Results

```
# Add standardised output columns
ranked['Candidate_ID'] = ranked['candidate_id']
ranked['match_%'] = (ranked['similarity_score'] * 100).round(2)
ranked.rename(columns={'predicted_category': 'Predicted_Category'},
inplace=True)

# Confirm final sort order
ranked = ranked.sort_values(by='similarity_score',
ascending=False).reset_index(drop=True)
ranked['rank'] = ranked.index + 1

print("Output columns configured.")
print(f"Total candidates ranked: {len(ranked)}")
display(ranked[['rank', 'candidate_id', 'similarity_score']])
```

Output columns configured.

Total candidates ranked: 2484

	rank	candidate_id	similarity_score
0	1	12011623	0.1867
1	2	18448085	0.1726
2	3	21156767	0.1462
3	4	18067556	0.1365
4	5	10265057	0.1344
...	...	...	...
2479	2480	32977530	0.0000
2480	2481	93002334	0.0000
2481	2482	24184357	0.0000
2482	2483	73077810	0.0000
2483	2484	17152392	0.0000

2484 rows × 3 columns

The following output presents the highest-ranked candidate and the top 7 candidates overall. Rankings are determined by cosine similarity between each candidate's resume and the defined job description.

```
# Display the best-matched candidate
best_candidate = ranked.iloc[0]

print("BEST CANDIDATE")
print("=" * 60)
print(f"Candidate ID      : {best_candidate['Candidate_ID']}")
print(f"Match Percentage    : {best_candidate['match_%']}%")
print(f"Predicted Category   : {best_candidate['Predicted_Category']}")
print("=" * 60)

BEST CANDIDATE
=====
Candidate ID      : 12011623
Match Percentage  : 18.67%
Predicted Category : ENGINEERING
=====

# Display Top 10 ranked candidates
print("TOP 10 RANKED CANDIDATES")
print("-" * 60)

display(
    ranked[['rank', 'Candidate_ID', 'match_%', 'Predicted_Category']].head(10)
)
```

TOP 5 RANKED CANDIDATES				
	rank	Candidate_ID	match_%	Predicted_Category
0	1	12011623	18.67	ENGINEERING
1	2	18448085	17.26	ENGINEERING
2	3	21156767	14.62	CONSULTANT
3	4	18067556	13.65	INFORMATION-TECHNOLOGY
4	5	10265057	13.44	INFORMATION-TECHNOLOGY
5	6	28505854	12.99	ENGINEERING
6	7	88907739	12.21	CONSULTANT
7	8	83816738	12.03	INFORMATION-TECHNOLOGY
8	9	30223363	10.76	INFORMATION-TECHNOLOGY
9	10	27152464	10.55	ENGINEERING

```

# — Plot top-15 ranked candidates

TOP_N_PLOT = 15

plot_df = (
    ranked[ranked['similarity_score'] > 0]
    .head(TOP_N_PLOT)
    .copy()
)

# Candidate Label: ID + category for readability
plot_df['label'] = (
    plot_df['candidate_id'].astype(str) + '\n' +
    plot_df['Predicted_Category'].str.title()
)

fig, ax = plt.subplots(figsize=(11, 7))

colors = plt.cm.RdYlGn(np.linspace(0.3, 0.9, len(plot_df))[:-1])

bars = ax.barh(
    plot_df['label'],
    plot_df['similarity_score'],
    color=colors, edgecolor='white', linewidth=0.8, height=0.6
)

for bar, val in zip(bars, plot_df['similarity_score']):
    ax.text(
        val + 0.001,
        bar.get_y() + bar.get_height() / 2,
        f'{val:.4f}',
        va='center', ha='left', fontsize=9, fontweight='bold'
    )

ax.set_xlabel('Cosine Similarity Score', fontsize=12)
ax.set_title(
    f'Top {TOP_N_PLOT} Candidates – Ranked by Relevance to Job Description',
    fontsize=13, fontweight='bold', pad=12
)

ax.set_xlim(0, plot_df['similarity_score'].max() * 1.2)
ax.invert_yaxis()
ax.spines[['top', 'right']].set_visible(False)
ax.tick_params(axis='y', labelsize=9)

plt.tight_layout()
plt.savefig(PLOTS_DIR / 'candidate_ranking.png', dpi=130, bbox_inches='tight')
plt.show()

```

```
print(f'Chart saved → {PLOTS_DIR / "candidate_ranking.png"}')
```

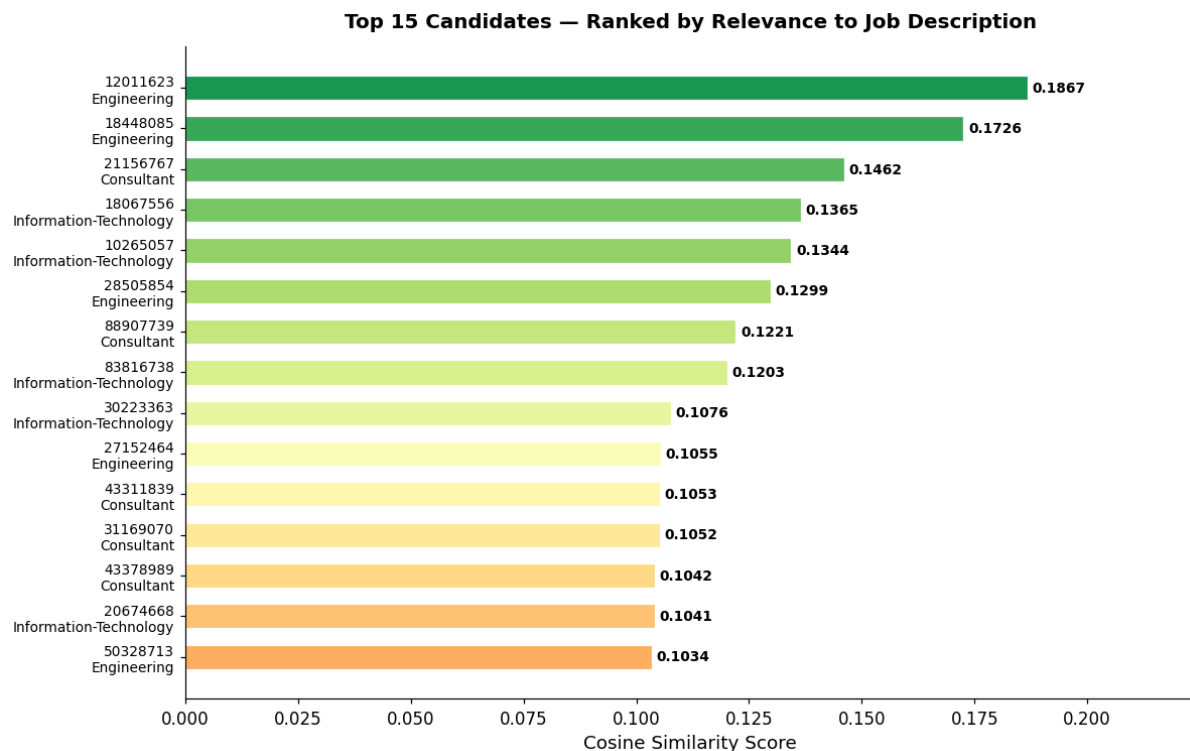


Fig. Top 15 candidates ranked by relevance to job description.

```
def explain_match(resume_text, job_description, top_k=15):
    """
    Show the top-K TF-IDF terms shared between a resume and a job description.
    Uses the ranking_vectorizer for consistency with the cosine similarity
    scores.
    """
    c_resume = preprocess_text(resume_text)
    c_jd      = preprocess_text(job_description)

    if not c_resume.strip() or not c_jd.strip():
        return pd.DataFrame({'error': ['One or both inputs empty after
preprocessing']})

    # Use ranking_vectorizer – same vectoriser used during cosine similarity
    vecs      = ranking_vectorizer.transform([c_resume, c_jd])
    resume_vec = np.asarray(vecs[0].todense()).flatten()
    jd_vec     = np.asarray(vecs[1].todense()).flatten()

    feature_names = ranking_vectorizer.get_feature_names_out()
    joint_score   = resume_vec * jd_vec # non-zero only where both have
weight

    # Filter to terms that actually appear in BOTH texts
```

```

shared_mask = joint_score > 0
if shared_mask.sum() == 0:
    return pd.DataFrame({'error': ['No shared terms found between resume
and JD']})

top_idx = joint_score.argsort()[::-1][:top_k]

return pd.DataFrame({
    'term'          : feature_names[top_idx],
    'resume_weight' : resume_vec[top_idx].round(4),
    'jd_weight'     : jd_vec[top_idx].round(4),
    'joint_score'   : joint_score[top_idx].round(6),
})
top_row          = ranked.iloc[0]
top_candidate_id = int(top_row['Candidate_ID'])

# Look up the actual resume text string from the original dataset
# (ranked.iloc[0] is a Series/row – not a text string)
top_resume_text = df[df['ID'] == top_candidate_id]['Resume_str'].values[0]

print('MATCH EXPLANATION – Top Shared Terms (Rank-1 Candidate vs JD)')
print('=' * 60)
print(f"Candidate ID : {top_candidate_id}")
print(f"Category      : {top_row['Predicted_Category']}")
print(f"Match %       : {top_row['match_%']}%")
print()

exp_df = explain_match(top_resume_text, job_description, top_k=15)
display(exp_df)

```

```

# Summary of ranking results
print("Ranking Summary")
print("-" * 60)
print(f"Total candidates evaluated      : {len(ranked)}")
print(f"Highest match percentage         : {ranked['match_%'].max()}%")
print(f"Lowest match percentage          : {ranked['match_%'].min()}%")
print(f"Average match percentage        : {ranked['match_%'].mean():.2f}%")
print()
print("Distribution of top 7 by predicted category:")
print(ranked.head(7)['Predicted_Category'].value_counts())

```

MATCH EXPLANATION – Top Shared Terms (Rank-1 Candidate vs JD)

=====

Candidate ID : 12011623
Category : ENGINEERING
Match % : 18.67%

	term	resume_weight	jd_weight	joint_score
0	machine learning	0.1272	0.2203	0.028019
1	python	0.1038	0.1799	0.018671
2	data analysis	0.0647	0.2352	0.015217
3	sql	0.1036	0.1279	0.013259
4	scientist	0.0555	0.2018	0.011199
5	data	0.0684	0.1604	0.010970
6	statistic	0.0633	0.1358	0.008592
7	statistical	0.0628	0.1349	0.008470
8	machine	0.0680	0.1179	0.008021
9	model	0.0612	0.1061	0.006492
10	learning	0.0629	0.0958	0.006028
11	analysis	0.0508	0.1120	0.005685
12	database	0.0686	0.0810	0.005557
13	finding	0.0352	0.1281	0.004515
14	communicate	0.0304	0.1104	0.003355

Ranking Summary

Total candidates evaluated : 2484
Highest match percentage : 18.67%
Lowest match percentage : 0.0%
Average match percentage : 0.73%

Distribution of top 7 by predicted category:

Predicted_Category
ENGINEERING 3
CONSULTANT 2
INFORMATION-TECHNOLOGY 2
Name: count, dtype: int64

5.1.8. Web Integration to Display Results:

The developed system integrates the machine learning model into a web-based application to provide an interactive and user-friendly interface for resume screening. The backend is implemented using **Flask**, which serves as a bridge between the trained model and the frontend interface. The trained vectorizer and model, obtained from the Jupyter Notebook, are serialized using pickle and loaded within the Flask application for real-time inference.

The frontend is designed using standard web technologies, including HTML, CSS, and JavaScript, ensuring a clean and responsive user interface. Users can input a job description and upload multiple resumes through the interface. Upon submission, the data is sent to the Flask server via an API endpoint, where preprocessing and vectorization are performed. The system then computes similarity scores using cosine similarity to evaluate the relevance of each resume with respect to the job description.

The processed results are returned in JSON format and dynamically rendered on the frontend. The interface displays ranked candidates along with their match scores, extracted skills, and explanatory insights. Additional features such as skill matching and result explanation enhance the transparency and usability of the system. This integration demonstrates the practical deployment of a machine learning model in a real-world web application environment.

Front End:

The front-end of the Automated Resume Screening System is designed to provide a simple and user-friendly interface for interacting with the underlying machine learning model. It is developed using **HTML and CSS**, ensuring ease of use, accessibility, and compatibility across different browsers.

The interface allows users to upload resumes, enter job descriptions, and view the processed results in a structured format.

The input section of the interface allows users to provide the necessary data for processing.

Resume Upload

- Users can upload one or multiple resume files (PDF/DOCX)
- Implemented using the HTML `<input type="file">` element
- Supports multiple file selection

Job Description Input

- A text area is provided for entering job requirements
- Allows flexible input of job-related details

Styling and Layout:

CSS is used to enhance the visual appearance and usability of the interface.

- Proper spacing and alignment for readability
- Use of buttons and input styling for better interaction
- Responsive layout to support different screen sizes

Integration with Backend:

The front-end communicates with the backend using **Flask**.

- Form data is sent using HTTP POST requests
- Backend processes resumes and returns results
- Results are rendered dynamically using templates

This integration ensures seamless data flow between user input and system output.

The web interface created using HTML, CSS and JS:

The screenshot shows the ResumeAI web interface. At the top, there's a logo and navigation links. The main heading is "Intelligent Resume Screening & Ranking" with a subtitle explaining the ML pipeline. Below this, there are four tabs: "TF-IDF Vectorisation", "LinearSVC Classifier", "Cosine Similarity", and "XAI Explanations". The "Analysis Configuration" section contains a "Job Description" text area with a character count and a "Resume Files" section with a file upload area and a "browse files" link. At the bottom, there's a blue button labeled "Analyse & Rank Candidates".

ResumeAI ML Powered NLP · TF-IDF · SVM

ML & NLP BASED

Intelligent Resume Screening & Ranking

Upload resumes and provide a job description. Our ML pipeline analyses content, extracts skills, computes cosine similarity and ranks candidates

TF-IDF Vectorisation LinearSVC Classifier Cosine Similarity XAI Explanations

Analysis Configuration

Job Description *

Paste the full job description here — the system will extract required skills and rank candidates by relevance...

0 characters

Include skills, responsibilities, and qualifications for best results.

Resume Files *

Drop resume files here
or [browse files](#)
.txt and .pdf supported · Multiple files allowed

★ Analyse & Rank Candidates →

Backend:

The backend of the Automated Resume Screening System is implemented using the Flask framework in Python. It acts as the core component responsible for handling user requests, processing input data, executing machine learning models, and returning the results to the front-end interface.

The backend integrates various modules including text preprocessing, feature extraction, classification, and similarity computation to ensure efficient and accurate resume screening.

Framework and Environment:

The backend is developed using the following technologies:

- Flask: Lightweight web framework for handling HTTP requests and routing
- Python: Core programming language for implementing logic and machine learning
- Scikit-learn: Used for model training, prediction, and similarity computation
- NLTK: Used for text preprocessing (tokenization, stopword removal, lemmatization)

This combination ensures flexibility, scalability, and ease of integration.

Request Handling and Routing:

Flask handles incoming requests from the front-end using defined routes.

Output Generation:

The backend prepares structured output data and sends it to the front-end.

- Results include:
 - Candidate name
 - Predicted category
 - Similarity score
- Data is passed to HTML templates using Flask rendering.

Error Handling

Basic validation and error handling mechanisms are implemented:

- Checks for empty file uploads
- Handles invalid file formats
- Prevents null input for job description

This ensures robustness and prevents runtime failures.

The user inputs the job description and all the resumes they want to rank. The system analyses the resumes and returns the best candidate.

In this case, this job description is used as the input:

We are looking for a Data Analyst to collect, process, and analyze data to help support business decisions. The candidate should be able to interpret data trends and provide actionable insights.

Key Responsibilities:

- *Analyze large datasets using statistical techniques*
- *Create reports and dashboards using tools like Excel, SQL, or Power BI*
- *Identify trends and patterns in data*
- *Work with teams to understand business requirements*

Required Skills:

- *Proficiency in SQL and Excel*
- *Basic knowledge of Python or R*
- *Experience with data visualization tools (Tableau/Power BI)*
- *Strong analytical and problem-solving skills*

Preferred Qualifications:

- *Bachelor's degree in Computer Science, Statistics, or related field*
- *Internship or project experience in data analysis*

We have also added 4 resumes to compute which is the best candidate for the job description.

Resume Files *



Drop resume files here

or [browse files](#)

.txt and .pdf supported · Multiple files allowed



SRIJANIDUTTA_CV.pdf

230.2 KB ×



alex_johnson.txt

757 B ×



john_smith.txt

1.4 KB ×



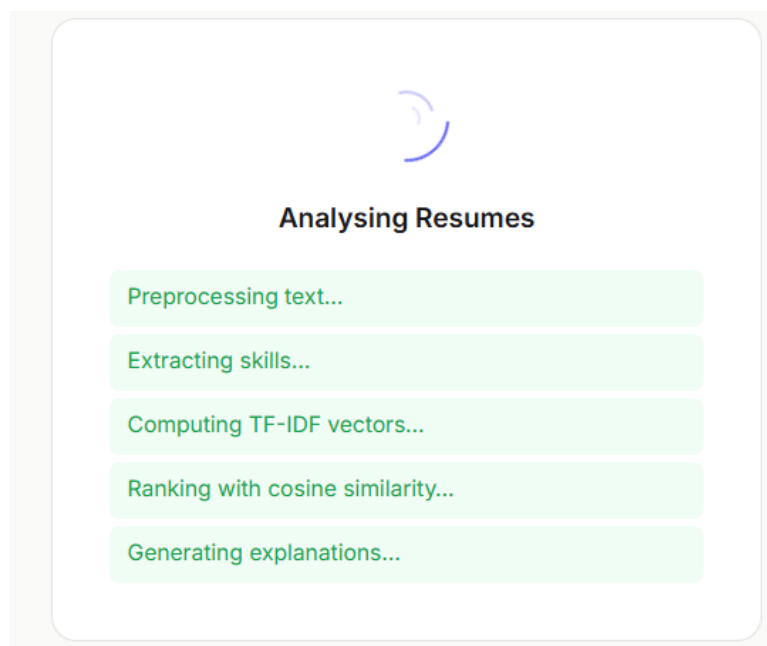
priya_sharma.txt

1.1 KB ×

★

Analyse & Rank Candidates

→



The final output showing the best matched candidate:

Candidate Rankings			← New Analysis
4 candidates analysed · Best match: 24.8%			
🏆 BEST MATCH john_smith.txt BUSINESS-DEVELOPMENT 24.8% MATCH SCORE john_smith.txt is ranked #1 with a low match score of 24.8%. Demonstrated strengths include: Data Analysis, Python, R, Sql, Statistics and 1 more skills. Skills that appear in the job description but are absent from this resume: Analytical, Data Visualization, Excel, Power Bi. This candidate's profile most closely aligns with the 'BUSINESS-DEVELOPMENT' domain. Data Analysis Python R Sql Statistics Tableau Analytical Data Visualization Excel Power Bi			
#3	alex_johnson.txt SALES	9.2% Match Score	9.2%
✓ MATCHED SKILLS Data Analysis Excel Sql ✗ MISSING SKILLS Analytical Data Visualization Power Bi Python R Statistics Tableau ▶ AI Explanation			
#4	priya_sharma.txt INFORMATION-TECHNOLOGY	4.8% Match Score	4.8%
✓ MATCHED SKILLS Sql ✗ MISSING SKILLS Analytical Data Analysis Data Visualization Excel Power Bi Python R Statistics Tableau ▶ AI Explanation			

Summary of methodology:

Stage	Technique Applied
Text Preprocessing	Regex cleaning, stopword removal, lemmatisation
Feature Extraction	TF-IDF with unigrams and bigrams (5,000 features)
Classification	Linear Support Vector Machine (LinearSVC)
Ranking	Cosine similarity in the TF-IDF vector space

Key outcomes:

- The trained Linear SVM classifier achieves strong accuracy on the held-out test set, demonstrating that TF-IDF features carry sufficient discriminative information to distinguish between professional resume categories.
- The ranking module successfully orders all candidates by relevance to a given job description, providing the recruiter with a prioritised shortlist without the need for manual screening.
- The system is dataset-driven, using the candidate ID field as the primary identifier throughout, ensuring that results can be traced directly back to individual records in the dataset.

5.2. Testing

Code Efficiency:

The efficiency of the system is ensured through optimized data processing and model implementation techniques. The use of TF-IDF vectorization with a limited feature size (5000 features) helps reduce computational complexity while maintaining relevant information. Additionally, the use of n-grams (unigrams and bigrams) improves feature representation without significantly increasing processing time.

Efficient data structures such as Pandas DataFrames and sparse matrices are utilized to handle large textual data effectively. The use of pre-trained models and model serialization (pickle) avoids repeated training, thereby reducing execution time during runtime.

Furthermore, preprocessing steps are designed to eliminate unnecessary data early in the pipeline, minimizing redundant computations. Overall, the system achieves a balance between accuracy and computational efficiency.

Testing Approach

Functional testing ensures that each module of the system performs its intended operation correctly, including preprocessing, classification, and ranking. User acceptance testing focuses on validating the usability of the web interface, ensuring that users can easily upload resumes, input job descriptions, and interpret the results.

Unit Testing

Unit testing is performed to verify the correctness of individual modules within the system. Each functional component is tested independently to ensure proper operation.

The following modules were tested:

- Preprocessing Module: Verified that text is correctly cleaned, tokenized, and normalized.
- Feature Extraction Module: Ensured that TF-IDF vectorization produces valid numerical representations.
- Classification Module: Tested machine learning models to confirm accurate prediction of resume categories.
- Similarity Computation Module: Verified that cosine similarity generates values within the expected range (0 to 1).

Unit testing ensures that each component functions correctly before integration into the complete system.

Integrated Testing

Integrated testing is performed after combining all modules of the system to ensure smooth interaction and overall functionality.

The following aspects were tested:

- End-to-end workflow from resume upload to result display
- Interaction between preprocessing, feature extraction, and ML models
- Integration of classification and similarity ranking modules
- Data flow between backend (Flask) and frontend (HTML/CSS)

The system was tested with multiple resumes and job descriptions to ensure consistent and reliable performance. No major errors or failures were observed during execution, indicating successful integration of all modules.

6. CONCLUSION

The Automated Resume Screening System presented in this project successfully demonstrates the application of Machine Learning and Natural Language Processing techniques in streamlining the recruitment process. The system is capable of analysing resumes, extracting relevant information, and matching them with job descriptions to identify suitable candidates.

The implementation integrates TF-IDF-based feature extraction, multiple classification models such as Naïve Bayes, Logistic Regression, and Linear SVM, and cosine similarity for ranking. The results indicate that the system performs effectively in classifying resumes and ranking candidates based on their relevance to job requirements. The web-based interface further enhances usability by allowing users to interact with the system in a simple and efficient manner.

Overall, the project achieves its objective of reducing manual effort, improving accuracy, and providing a scalable solution for automated resume screening.

6.1. Limitations of the System:

While the proposed Automated Resume Screening System demonstrates effective performance, it is subject to certain limitations:

1. **Dependence on Dataset Quality:** The accuracy of the system is directly influenced by the quality, size, and diversity of the training dataset. A limited or imbalanced dataset may lead to biased predictions and reduced generalization.
2. **Limited Semantic Understanding:** The system uses TF-IDF for feature extraction, which primarily captures keyword frequency and does not fully understand contextual meaning, synonyms, or semantic relationships between words.
3. **Handling of Multi-domain Resumes:** Resumes containing skills from multiple domains may not be accurately classified into a single category, leading to potential misclassification.
4. **Keyword Dependency:** The system relies heavily on explicit keywords present in resumes and job descriptions. Candidates with relevant skills described in different terminology may not be ranked accurately.

5. **Lack of Contextual Evaluation:** The system does not consider deeper contextual factors such as project quality, work experience relevance, or soft skills, which are important in real-world recruitment.
6. **Limited File Format Support:** The current implementation primarily supports PDF and DOCX formats. Other formats or poorly structured documents may not be processed effectively.
7. **Scalability Constraints:** While suitable for small to medium datasets, performance may degrade when handling very large volumes of resumes without further optimization or distributed processing.

6.2. Future Scope of the Project

The system can be significantly enhanced through the following improvements:

1. **Advanced NLP Techniques:** Incorporating advanced models such as word embeddings (Word2Vec, GloVe) or transformer-based architectures like BERT can improve semantic understanding and capture contextual meaning more effectively.
2. **Improved Feature Engineering:** Additional features such as experience weighting, skill importance scoring, and domain-specific keyword extraction can enhance the accuracy of candidate evaluation.
3. **Multi-label Classification:** Instead of assigning a single category, the system can be extended to support multi-label classification, allowing resumes to belong to multiple domains simultaneously.
4. **Enhanced Ranking Mechanism:** A more sophisticated ranking system can be implemented by combining similarity scores with additional factors such as years of experience, education level, and project relevance.
5. **Integration with Real-world Systems:** The system can be integrated with job portals, Applicant Tracking Systems (ATS), or HR management platforms to enable end-to-end automation of the recruitment process.
6. **User Interface Improvements:** The web interface can be enhanced with advanced features such as filtering, sorting, visualization of scores, and interactive dashboards for better user experience.
7. **Cloud Deployment and Scalability:** Deploying the system on cloud platforms can improve scalability, allowing it to handle large-scale recruitment scenarios efficiently.
8. **Automated Resume Parsing:** More advanced parsing techniques can be implemented to extract structured information such as skills, certifications, and experience more accurately.

9. **Incorporation of Feedback Mechanism:** A feedback loop can be introduced where recruiters can validate results, enabling the system to improve over time through continuous learning.
10. **Bias Reduction and Fairness:** Future improvements can focus on reducing bias in model predictions to ensure fair and unbiased candidate evaluation.

The Automated Resume Screening System successfully demonstrates the use of Machine Learning and Natural Language Processing techniques to automate the recruitment process. It effectively classifies and ranks resumes based on their relevance to job descriptions, reducing manual effort and improving efficiency. The integration of classification models and similarity-based ranking ensures accurate and practical candidate evaluation. Overall, the system provides a scalable and reliable solution for modern hiring processes, with potential for further enhancement.

REFERENCES

1. S. T. Al-Otaibi and M. Ykhlef, "A survey of job recommender systems," *International Journal of Physical Sciences*, vol. 7, pp. 5127–5142, 2012.
2. J. A. Breagh, "The use of biodata for employee selection: Past research and future directions," *Human Resource Management Review*, vol. 19, pp. 219–231, 2009.
3. L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
4. W. Carrer-Neto, M. L. Hernández-Alcaraz, R. Valencia-García, and F. García-Sánchez, "Social knowledge-based recommender system: Application to the movies domain," *Expert Systems with Applications*, vol. 39, pp. 10990–11000, 2012.
5. O. Celma, "Music recommendation," in *Music Recommendation and Discovery*. Springer, 2010, pp. 43–85.
6. A. S. Das, M. Datar, A. Garg, and S. Rajaram, "Google news personalization: Scalable online collaborative filtering," in *Proc. 16th Int. Conf. World Wide Web*, 2007, pp. 271–280.
7. Q. Diao, M. Qiu, C. Y. Wu, A. J. Smola, J. Jiang, and C. Wang, "Jointly modeling aspects, ratings and sentiments for movie recommendation (JMARS)," in *Proc. 20th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2014, pp. 193–202.
8. F. Färber, T. Weitzel, and T. Keim, "An automated recommendation approach to selection in personnel recruitment," in *Proc. AMCIS*, 2003, p. 302.
9. A. Golec and E. Kahya, "A fuzzy model for competency-based employee evaluation and selection," *Computers & Industrial Engineering*, vol. 52, pp. 143–161, 2007.
10. J. L. Howard and G. R. Ferris, "The employment interview context: Social and situational influences on interviewer decisions," *Journal of Applied Social Psychology*, vol. 26, pp. 112–136, 1996.
11. Y. Lin, H. Lei, P. C. Addo, and X. Li, "Machine learned resume-job matching solution," *arXiv preprint arXiv:1607.07657*, pp. 1–8, 2016.
12. E. Loper and S. Bird, "NLTK: The natural language toolkit," *arXiv preprint cs/0205028*, 2002.
13. J. Lu, D. Wu, M. Mao, W. Wang, and G. Zhang, "Recommender system application developments: A survey," *Decision Support Systems*, vol. 74, pp. 12–32, 2015.
14. S. Maheshwary and H. Misra, "Matching resumes to jobs via deep Siamese network," in *Proc. Companion Web Conf.*, 2018, pp. 87–88.
15. J. Malinowski, T. Keim, O. Wendt, and T. Weitzel, "Matching people and jobs: A bilateral recommendation approach," in *Proc. 39th Annu. Hawaii Int. Conf. System Sciences (HICSS)*, 2006, pp. 137c–137c.

16. R. J. Mooney and L. Roy, "Content-based book recommending using learning for text categorization," in *Proc. 5th ACM Conf. Digital Libraries*, 2000, pp. 195–204.
17. N. M. Nasrabadi, "Pattern recognition and machine learning," *Journal of Electronic Imaging*, vol. 16, p. 049901, 2007.
18. I. Paparrizos, B. B. Cambazoglu, and A. Gionis, "Machine learned job recommendation," in *Proc. 5th ACM Conf. Recommender Systems*, 2011, pp. 325–328.
19. J. Ramos *et al.*, "Using TF-IDF to determine word relevance in document queries," in *Proc. 1st Instructional Conf. Machine Learning*, Piscataway, NJ, 2003, pp. 133–142.
20. P. Resnick and H. R. Varian, "Recommender systems," *Communications of the ACM*, vol. 40, pp. 56–59, 1997.
21. I. Rish *et al.*, "An empirical study of the naive Bayes classifier," in *Proc. IJCAI Workshop on Empirical Methods in Artificial Intelligence*, 2001, pp. 41–46.
22. P. K. Roy and J. P. Singh, "A tag2vec approach for question tag suggestion on community question answering sites," in *Proc. Int. Conf. Machine Learning and Data Mining in Pattern Recognition*, 2018, pp. 168–182.
23. P. K. Roy, J. P. Singh, A. M. Baabdullah, H. Kizgin, and N. P. Rana, "Identifying reputation collectors in community question answering sites: Exploring the dark side of social media," *International Journal of Information Management*, vol. 42, pp. 25–35, 2018.
24. P. K. Roy, J. P. Singh, and A. Nag, "Finding active expert users for question routing in community question answering sites," in *Proc. Int. Conf. Machine Learning and Data Mining in Pattern Recognition*, 2018, pp. 440–451.
25. J. B. Schafer, J. Konstan, and J. Riedl, "Recommender systems in e-commerce," in *Proc. 1st ACM Conf. Electronic Commerce*, 1999, pp. 158–166.
26. B. Schölkopf, A. J. Smola, and F. Bach, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Cambridge, MA, USA: MIT Press, 2002.
27. J. P. Singh, S. Irani, N. P. Rana, Y. K. Dwivedi, S. Saumya, and P. K. Roy, "Predicting the helpfulness of online consumer reviews," *Journal of Business Research*, vol. 70, pp. 346–355, 2017.
28. K. Wei, J. Huang, and S. Fu, "A survey of e-commerce recommender systems," in *Proc. Int. Conf. Service Systems and Service Management*, 2007, pp. 1–5.
29. X. Yi, J. Allan, and W. B. Croft, "Matching resumes and jobs based on relevance models," in *Proc. 30th Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2007, pp. 809–810.
30. L. Zhang, W. Fei, and L. Wang, "PJ matching model of knowledge workers," *Procedia Computer Science*, vol. 60, pp. 1128–1137, 2015.